

Betriebssysteme

2 – ÜB – Prozessverwaltung

Aufgabe 1: Grundkonzepte

- a) Ein Kollege argumentiert: "Dass ein Browser für jeden offenen Tab einen eigenen Prozess anlegt, ist gar nicht so gut: dann kann nämlich nur ein einzelner Tab etwas tun (z. B. Musik wiedergeben)." Der Kollege hat nicht Recht. Warum? Geben Sie zudem an, was für Vorteile und Nachteile ein Prozess pro Tab wirklich hat.

Moderne Betriebssysteme können auch mehrere Prozesse gleichzeitig oder quasi-gleichzeitig laufen lassen. Es ist eine häufige Fehlvorstellung, ein OS könne "nur" Threads parallel laufen lassen. In der konkreten Realisierung sieht es meist so aus, dass jeder Prozess gleichzeitig als zumindest ein Thread modelliert wird, der dann (zusammen mit allen anderen Threads) vom Prozessscheduler geplant wird.

Vorteile:

- **besserer Speicherschutz der Prozesse gegeneinander (vgl. VL-Einheit 3), ein korruptiertes Skript aus einer Tab kann dadurch z. B. nur schwer direkt Daten wie ein eingegebenes Passwort aus einer anderen Tab auslesen**
- **falls eine Tab (z. B. durch falschen Speicherzugriff, vgl. VL-Einheit 3) abstürzt, wird der zugehörige Prozess vom Betriebssystem abgeschossen, der restliche Browser kann weiterlaufen -- wäre dagegen der Browser insgesamt nur ein Prozess, würde das OS in diesem Fall den ganzen Browser beenden**

Nachteile:

- **etwas mehr Speicherkosten pro offener Tab (z. B. für die Seitentabelle, vgl. VL-Einheit 3)**
- **deutlich mehr Laufzeitkosten, um Daten zwischen der Tab und dem Browser zu teilen, z. B.: wie kommt das anzuzeigende "Gesamtbild" der Webseite vom Prozess des Tabs in die GUI-Tab hinein -- die ja weiterhin zum "Hauptthread" des Browsers gehört? -> langsames Kopieren zwischen Prozessen oder andere, etwas schnellere Tricks**

- b) Kann ein Rechner mit einem Kern mehrere Programme so ausführen, dass ein Endnutzer mit diesen Programmen gleichzeitig arbeiten kann? Falls ja: Wie? Falls nein: Warum nicht?

Hinweis: Sie dürfen hier gerne Wortklauberei betreiben!

Gleichzeitig "im engeren Sinne" geht das nicht, da ein Prozessor mit einem Kern exakt eine Instruktion von exakt einem Programm zu jedem Zeitpunkt ausführen kann. Wenn der Prozessor aber sehr schnell (z. B. in der Praxis oft im Takt von ~10ms) zwischen mehreren, laufenden Programmen hin- und herspringt (z. B. von jedem Programm ein paar Instruktionen rechnet) und einige weitere Tricks einsetzt (z. B. Pufferung, vgl. VL-Einheit 5), fällt dem Benutzer

nicht auf dass zu jedem Zeitpunkt nur ein konkretes Programm gerade läuft. Für den Benutzer sieht es stattdessen so aus, als würden alle Programme gleichzeitig laufen. Man spricht daher von "Quasi-Parallelität".

c) Was ist im Rahmen eines Prozesswechsels konkret für das Betriebssystem zu tun?

Das Betriebssystem kennt an dieser Stelle schon den nächsten und den alten Prozess (vgl. Scheduling der VL).

Was ist zu tun (in unbestimmter Reihenfolge aufgeführt):

- Abarbeitung des aktuellen Programms "von außerhalb der CPU" stoppen lassen (z. B. mit Unterbrechungswerk, vgl. VL-Einheit 4) und eine OS-Funktion für den Prozesswechsel auf die CPU legen, die alle nachfolgenden Schritte ausführt

- Inhalte der Prozessorregister (z. B. nächste Maschinencode-Instruktion, alle Variablen, auf denen gerechnet wird) des alten Prozesses in den Hauptspeicher sichern

- OS-interne Datenstrukturen zur Prozessverwaltung anpassen (z. B. alten Prozess in die Blockiert- oder Bereit-Warteschlange setzen, nächsten Prozess aus Warteschlange lösen und als "laufend" markieren)

- Kern für den neuen Prozess vorbereiten: Register auf lokale Variablen des neuen Prozesses (z. B. aus vorheriger Sicherung in den Hauptspeicher) setzen und Instruktionszeiger auf die nächste Anweisung des nächsten Prozesses setzen

- eventuell weitere Verwaltungsaufgaben (z. B. Seitentabelle anpassen, E/A-Operationen anhalten oder starten, Prozessrechte setzen, ...)

- Unterbrechung beenden und die CPU weiterrechnen lassen -> die nächste Instruktion des nächsten Prozesses wird ausgeführt

Hinweis: Threadwechsel funktioniert analog. Meist sind die Scheduler ohnehin nur für "Threads" implementiert, jeder Prozess besteht dann zumindest aus einem "Thread" und geht so in das Scheduling ein.

d) Wie muss eine Anwendung einen Prozesswechsel im Code berücksichtigen?

Hinweis: Woran merkt eine Anwendung, dass der zugehörige Prozess oder ein Thread vom Betriebssystem verdrängt wird oder auf blockiert schaltet?

Meist: gar nicht. Es ist ja nicht so, als würde jedes "Programm" alle paar Zeilen auf eine Verdrängung prüfen müssen. Eine laufende Anwendung kann vom OS an (fast) jeder Stelle für einen Prozesswechsel zu einer anderen, laufenden Anwendung unterbrochen werden.

anders ausgedrückt: Programme sehen ja NICHT so aus:

```
void do_something()
{
    if (jetzt_prozess_wechseln) lass_os_prozess_wechseln();
    mache_was_schritt_1();
    if (jetzt_prozess_wechseln) lass_os_prozess_wechseln();
    mache_was_schritt_2();
    if (jetzt_prozess_wechseln) lass_os_prozess_wechseln();
    mache_was_schritt_3();
}
```

sondern eher so:

```
void do_something()
{
    mache_was_schritt_1();    // evtl. zwischendrin Prozesswechsel möglich
    mache_was_schritt_2();    // evtl. zwischendrin Prozesswechsel möglich
    mache_was_schritt_3();    // evtl. zwischendrin Prozesswechsel möglich
}
```

Konkret gibt es auch keinen konkreten Systemaufruf, um festzustellen, ob denn der eigene Prozess "vor kurzem" gewechselt wurde (und während der Prozess nicht auf der CPU läuft, kann er ohnehin keine Funktionen aufrufen). Man kann sich natürlich mit Tricks behelfen, z. B. regelmäßig die Systemuhr abfragen -- gab es zwischenzeitig einen Prozesswechsel, sieht man das daran, dass die Aufrufe weiter auseinanderliegende Zeiten angeben (z. B. 10 ms statt 1ms Abstand).

- e) Geben Sie drei (konzeptuell unterschiedliche) Gründe dafür an, dass ein Thread vom Aktiv-Zustand in den Blockiert-Zustand wechselt.

der Thread wartet auf eine Rechenresource AUßER der CPU (z. B. der Thread möchte einen bereits belegten Mutex selbst belegen oder eine bereits geöffnete Datei abfragen) -- ACHTUNG: das Warten nur auf die CPU (wenn sonst alle anderen Ressourcen da sind) führt zum Bereit-Zustand.

der Thread wartet auf den Abschluss eines länger laufenden Systemaufrufs (z. B. Abholen von Daten von einem Peripheriegerät wie der Festplatte oder der Netzwerkkarte, vgl. VL-Einheit 4)

der Thread hat sich selbst für eine begrenzte Zeit schlafen gelegt (z. B. über einen Sleep-Befehl), weil erwartungsgemäß in nächster Zeit kein Bedarf zum Rechnen besteht -- die meisten Anwendungen machen etwas in dieser Art, um Strom zu sparen (sonst wäre immer 100% CPU-Auslastung, wenn immer ein Prozess rechnen müsste)

- f) Was ist damit gemeint, dass ein Thread von der CPU verdrängt wird? Warum landet der Thread dann im Bereit- statt im Aktiv-Zustand?

Ein Thread wird von der CPU verdrängt, weil das OS per Scheduling und einer internen Regel (z. B. Heuristik) beschließt, dass jetzt ein anderer Thread auf der CPU rechnen darf.

Verdrängt heißt konkret: Dem laufenden Thread wird die CPU entzogen, obwohl der Thread eigentlich weiter rechnen könnte (deswegen kein Blockiert-Zustand sondern Bereit-Zustand als neuer Zustand).

Das OS verdrängt Threads, damit jeder Thread "mal" an die Reihe kommt und damit ein einzelner Thread mit viel zu rechnen nicht alle anderen Threads von der CPU abhält -- insgesamt also, damit das OS reaktiv bleibt und damit der Benutzer "Quasi-parallelität" aller (d. h. hunderter) Threads und Prozesse bekommt.

- g) Warum braucht jeder Thread einen eigenen Stapel?

Mehrere Threads eines Programms arbeiten in der Regel zur gleichen Zeit an sowie in verschiedenen Funktionen des Programms -- z. B. könnte Thread A gerade in Funktion f() stehen, die aus Funktion g() aufgerufen wurde, während Thread B in Funktion h() steht, die aber ebenfalls aus Funktion g() aufgerufen wurde. Beide Threads müssen sich EIGENE lokale Variablen merken können -- und zwar Thread A für Funktionen f() und g(), Thread B für Funktionen h() und g(). Man braucht also für jeden Thread eventuell einige lokale Variablen, die andere Threads nicht brauchen (im Beispiel: h() vs. f()) und man braucht für jeden Thread eigene Kopien von lokalen Variablen, die eventuell in anderen Threads ebenfalls (als eigene Kopie) bestehen (im Beispiel: Variablen in g()). Das ist über einen eigenen Stapel pro Thread zu erreichen.

ACHTUNG: Damit sich Threads Variablen teilen können, braucht man ein paar Tricks -- z. B. müssen die Variablen auf dem Heap liegen, oder man muss eine zu teilende Stapelvariable per Verweis (Referenz in C#, Zeiger in C++) von einem Thread zu allen anderen Threads weitergeben. Letzteres ist der Trick aus dem Beispiel zu Race Conditions an der VL: die geteilte Variable wird vom Hauptthread auf dessen Stapel angelegt und von dort per Zeiger an die anderen Threads direkt (d. h. NICHT als Kopie) weitergegeben.

- h) Angenommen, es gibt mehr Kerne als Threads. Warum gibt es dann keine Threads, die (mehr als sehr kurzfristig) im Bereit-Zustand sind?

Weil es für jeden Thread dann mindestens einen Kern gibt, auf dem der Thread rechnen kann -- das wäre dann ein "echter" Multiprogrammbetrieb mit vollständig "echter" Parallelität statt Quasiparallelität. Sobald also ein Thread in den Bereitzustand wechselt, steht für den Thread sofort ein Kern zur Verfügung und der Thread wechselt gleich in den Aktiv-Zustand weiter.

- i) Warum führt die Bevorzugung kurzer Prozesse zu einer kürzeren mittleren Bearbeitungszeit sowie zu geringerer Auslastung? Welche Annahmen stecken in dieser Argumentation?

Durch das Zwischenschieben "kurzer" Prozesse zum frühestmöglichen Zeitpunkt hält man diese Prozesse davon ab, im Vergleich zu ihrer kurzen Laufzeit unnötig viel Wartezeit anzusammeln, so dass deren Bearbeitungszeit jeweils so kurz wie möglich bleibt. Lange Prozesse werden durch "etwas" zusätzliche Wartezeit nicht deutlich länger, als sie ohnehin schon sind. Da lange und kurze Prozesse in der durchschnittlichen Bearbeitungszeit gleich gewichtet eingehen, sinkt diese damit. Allerdings hat man dann für lange Prozesse nichts mehr übrig, was man "gegen Ende" des langen Prozesses nutzen kann, um eventuelle Blockiert-Zeiten des langen Prozesses mit sinnvollen Rechnungen zu füllen -> die CPU-Auslastung des Systems nimmt gegen Ende der Gesamtzeit ab.

- j) Probieren Sie die Codebeispiele zu Threads (F26, F27, F30, F31, F32, F33, F35, F40) auf einem echten System aus! Achten Sie besonders darauf, ob die genannten Fehler überhaupt und konsistent zu reproduzieren sind.

hier ergibt eine Musterlösung keinen Sinn

Praxishinweis: wenn kein Fehler passiert, kann es unter anderem an der Programmiersprache liegen -- manche Programmiersprachen haben z. B. von sich aus threadsichere Strings

- k) Liefert First-Come-First-Serve eine eindeutige Planungsreihenfolge? Argumentieren Sie an einem Beispiel.

Nein, denn es kann ja sein, dass auf zwei aktiven Kernen zeitgleich aktive zwei Prozesse auch zeitgleich in den Blockiert-Zustand wechseln. An dieser Stelle muss das Betriebssystem eine Reihenfolge in der Warteschlange festhalten (z. B. nach Prozess-Nummer), diese Reihenfolge steht aber nicht mehr im Rahmen einer FIFO-Planung fest.

- l) Gegeben seien die folgenden Prozesse mit [A]ktiv- und [B]lockier-Zeiten (z. B. in Sekunden angegeben):

- Prozess 1: A3, B3, A3, B3, A1
- Prozess 2: A2, B4, A2
- Prozess 3: A5, B1, A5, B1, A5, B1, A1

Die Deadline für Prozess 1 ist 15, für Prozess 2 ist die Deadline 14, und für den Prozess 3 ist die Deadline 16.

Führen Sie eine Planung mit dem Earliest-Deadline-First-Verfahren durch und zeichnen Sie das Ergebnis. Werden alle Deadlines eingehalten?

Annahme: das System hat nur einen Kern und Prozesswechsel finden nur statt, wenn ein Prozess blockiert wird (d. h. kein preemptives Multitasking)

Durch EDF wird P2 über P1 und P1 über P3 bevorzugt.

Damit ergibt sich die folgende Kernbelegung: --> Sekunden

P1 WW AAA BBBWW WW AAA BBBWW A|

P2 AA BBB BWWWW AA|

P3 WW WWW AAAAA BW WWW AAAAA B AAAAA B A|

W steht für "Warten" im Bereitzustand

A ist Aktiv-Zustand, Prozess rechnet auf dem Kern

B ist Blockiert-Zustand, Prozess wartet z. B. auf Resource

P1 und P3 halten ihre Deadline nicht ein. Ein Einhalten aller Deadlines ist in diesem Fall grundsätzlich nicht möglich, weil P3 schon mitsamt Blockiert-Zeiten seine Deadline nicht einhalten kann, selbst wenn P3 durchgehend die CPU bekommt.

m) Warum sind im vorangehenden Beispiel keine Bereit-Zeiten angegeben?

Die Bereit-Zeiten kann man nicht an einem konkreten Thread oder Prozess selbst festmachen, sondern die Bereit-Zeiten hängen davon ab, wie das OS die Kernzuteilung macht -- d. h. die Bereit-Zeiten sind (nur) vom Scheduling-Verfahren abhängig, die Aktiv- und Blockiert-Zeiten sind (nur) vom Prozessverhalten abhängig

n) Betrachten Sie Race Conditions und Deadlocks für Prozesse und Dateien:

- Gehen Sie davon aus, dass das OS von einem Prozess geöffnete Dateien nicht für andere Prozesse sperrt. Was müssen Prozesse machen, dass es eine Race Condition gibt und wie äußert sich die Race Condition?
- Gehen Sie davon aus, dass das OS von einem Prozess geöffnete Dateien für andere Prozesse sperrt. Löst das das Problem aus dem vorigen Punkt? Wieso und was passiert?
- Gehen Sie immer noch davon aus, dass das OS geöffnete Dateien sperrt. Wie kann es in dieser Situation zu einem Deadlock kommen? Was muss welcher Prozess in welcher Reihenfolge tun?

P1: die Prozesse müssen zeitgleich die selbe Datei (z. B. selber Pfad, selber Dateiname) öffnen und dort etwas hineinschreiben. Je nachdem, wie genau das "Schreiben" realisiert ist, kann sich die Race Condition in diesem Fall auf verschiedene Arten äußern -- es kann eine Datei mit "Mischinhalt" aus beiden

Schreibbefehlen entstehen bis hin zu Hardware-Problemen (z. B. weil dann der Festplattencontroller über den Bus fehlerhafte Daten erhält).

Beispiel: Prozess A schreibt "Frühstück", Prozess B schreibt "Mittagessen", Ergebnis könnte z. B. sein "FrMitühstütageckssen" oder "Frühagesken".

P2: Ja, das Problem der Race Condition ist gelöst. Ein Prozess erhält dann beim Öffnen einer Datei eine Fehlermeldung vom Betriebssystem zurück (etwa könnte open() einen NULL-Wert statt einen Dateihandle zurückgeben), falls die Datei schon von einem anderen Prozess geöffnet ist. Intern führt das Betriebssystem eine geeignete Synchronisation durch (z. B. per Mutex).

P3: Es kommt zu einem Deadlock, wenn auch hier die 4 Bedingungen erfüllt sind. Exklusivität ist laut Angabe schon erfüllt. Externe Freigaben gibt es nicht, weil nur ein Prozess selbst (per close()) eine Datei wieder freigeben kann und weil das kann einem Prozess in der Regel nicht mittendrin die Datei wegnehmen kann, ohne dass es zu Fehlern in der Anwendung (halb geschriebene Dateien) gibt. Wir brauchen demnach noch: eine Zusatzbelegung und einen Zyklus.

Vorgehen: Prozess A hat Datei A geöffnet und macht während dieser Zeit eine Anfrage ans OS, zusätzlich Datei B zu öffnen. Prozess B hat Datei B geöffnet und macht während dieser Zeit eine Anfrage ans OS, zusätzlich Datei A zu öffnen (ab jetzt gibt es einen Zyklus). An der Zusatzbelegung muss man noch etwas feilen: Wenn das OS aus open() einen Fehler zurückgibt, statt bei open() zu warten, bis die Datei verfügbar ist, dann findet keine zwingende Zusatzbelegung statt. Die Prozesse müssen also zusätzlich in einer endlosen while-Schleife warten, bis sie die Datei tatsächlich geöffnet bekommen, damit eine Zusatzbelegung vorliegt. Damit sind dann alle 4 Bedingungen erfüllt und es gibt ein Deadlock.

- o) Wieso ist das Scheduling dafür verantwortlich, dass Race Conditions schwierig zu reproduzieren sind? Was muss man am Scheduling (und gegebenenfalls an den Prozessen) ändern, damit Race Conditions sicher reproduzierbar werden?

Grundproblem ist, dass das Scheduling ebenfalls nicht reproduzierbar ist: Das OS legt nach einer (für normale Entwickler unbekannten) Heuristik fest, welche Threads wann und in welcher Reihenfolge auf der CPU rechnen. Selbst wenn man die Heuristik kennt, ist es in der Regel immer noch nicht möglich, konkret vorherzusagen, wann ein Thread rechnet, da das Scheduling weiter von allen laufenden Threads und deren aktuellen Aktivitäten abhängt (z. B. eingehende Netzwerkpakete verursachen eine unvorhersehbare Blockiert- oder Aktiv-Reihenfolge eines Threads einer Netzwerkanwendung, der damit eine immer wieder andere Verdrängung der auf Race Conditions zu untersuchenden Threads zur Folge hat).

Man muss also für reproduzierbare Race Conditions die Möglichkeit haben, das Scheduling ebenfalls reproduzierbar zu gestalten, z. B. indem man eine feste Reihenfolge aller CPU-Belegungen angibt. Das geht aber nur in sehr

speziellen Anwendungen (z. B. Steuergeräte für Autos, Flugzeuge, ...)

- p) Erklären Sie für jedes der 4 Deadlock-Kriterien, warum kein Deadlock mehr möglich ist, wenn das Kriterium nicht mehr erfüllt ist (aber alle anderen Kriterien weiterhin erfüllt sind).

Nicht exklusive Mutexe: Wenn bei mehreren Belegungen ein Zyklus mit Zusatzbelegung und ohne externe Freigabe auftritt, dann ist die zyklische Belegung trotzdem erfolgreich statt endlos zu warten, da ein Mutex jetzt von mehreren Beteiligten beansprucht werden kann. Dummerweise erfüllt der Mutex damit seine Funktion nicht mehr.

Zusatzbelegung: Ohne Zusatzbelegung kann kein Zyklus auftreten, sondern jeder Thread belegt zu einer Zeit höchstens eine Resource exklusiv und ohne eine externe Freigabe. Damit gibt es nur zwei Fälle: entweder der Thread bekommt den Mutex und kann damit seine Aufgabe vollenden und den Mutex irgendwann wieder freigeben, oder der Thread bekommt den Mutex nicht -- dann wird der Mutex vom Besitzer aber nach vorheriger Argumentation wieder freigegeben und es gibt kein ewiges Warten.

Zyklus: Angenommen es gibt exklusive Mutexe, keine externen Freigaben und Zusatzbelegungen, aber keine Zyklen. Dann gibt es einen konkreten Thread, der eine letzte, innerste Zusatzbelegung vornimmt, ohne dass der zugehörige Mutex schon belegt ist (denn das wäre ja ein ausgeschlossener Zyklus). Damit kann dieser konkrete Thread seine Arbeit erledigen, dann alle Threads, die auf Ressourcen dieses Threads gewartet haben, dann alle Threads, die auf deren Ressourcen gewartet haben, und so weiter.

Externe Freigaben: Angenommen, es gibt exklusive Mutexe, Zusatzbelegungen und Zyklen. Dann kann sich eine deadlock-artige Situation einstellen: Thread A hat Mutex A und wartet auf Mutex B, Thread B wartet auf Mutex A und hält Mutex B. In einer solchen Situation gibt es mit der externen Freigabe die Möglichkeit, dass ein außenstehender Agent (z. B. das OS) die Verklemmung erkennt und die externe Freigabe auslöst: Alle Threads werden (z. B. mit einer Fehlermeldung) aus den Mutexen gelöst und können darauf entsprechend reagieren. Eine typische Lösung wäre es, alle Operationen neu zu versuchen und zu hoffen, dass dieses Mal kein Deadlock auftritt. Damit ist das "zwingende" Deadlock entschärft. Achtung: Das Verfahren ist nur probabilistisch deadlockfrei, im Sinne: Wenn man das Programm nur lange genug laufen lässt, so laufen die Operationen irgendwann einmal vollständig durch, müssen aber mit etwas Pech dafür sehr oft neu gestartet werden.

- q) Bei einem Lese-Schreiber-Problem versuchen mehrere Leser und Schreiber, auf die selbe Variable zuzugreifen. Geben Sie die Einschränkungen wieder, die bei Leser-/Schreiber-Problemen für den Zugriff gelten (Wann darf ein Schreiber schreiben? Wann darf ein Leser lesen?).

Beschreiben Sie für Leser und Schreiber jeweils das Problem, das beim Verzicht auf diese Einschränkungen entstehen kann.

mehrere Leser: das ist in aller Regel kein Problem, hier braucht man mehrere, konkurrierende Leser nicht gegeneinander absichern (z. B. kein Mutex nötig)

sobald ein Schreiber dazukommt: jetzt dürfen keine Leser die Variable mehr lesen und kein anderer Schreiber ist zulässig -- jetzt braucht man eine geeignete Absicherung (z. B. Mutex)

sobald man die Absicherung im letzteren Fall weglässt gibt es zwei klassische Ausprägungen einer Race Condition:

- Leser lesen eventuell ein halb-vollständiges Ergebnis (z. B. eine nur teilweise geänderte Datenstruktur)

- Schreiber überschreiben sich gegenseitig die Änderungen, so dass eine Mischung oder ein vollständig kaputtes Ergebnis entsteht (z. B. eine kaputt geschriebene Datenstruktur)

- r) Die Abfrage der Variable `value_` hat einen Bug. Welchen? Wann und wie äußert sich ein Problem in diesem Zusammenhang?

```
float get_value()
{
    std::lock(this->mutex_);
    return this->value_;
}
```

Dieser Bug ist sehr klassisch: der Mutex wird zwar belegt, aber nicht mehr freigegeben. Damit wird jeder weitere Aufruf von `get_value()` aus einem anderen Thread endlos auf die Freigabe warten und das Programm friert ein. Daher haben viele Sprachen eine Möglichkeit, um Freigaben automatisch am Ende eines Bereichs (engl. Scope, `{ }`-Block) durchzuführen, z. B. mit einem eigenen Schlüsselwort (`synchronized()` in Java, `using` in C#) oder mit einem Programmierparadigma (RAII, Konstruktoren, Destruktoren in C++).

- s) Schreiben Sie eine Anwendung in einer Sprache Ihrer Wahl für folgendes Szenario: Zwei Threads füllen eine FIFO-Schlange mit Strings der Art "Thread [Threadnummer] -- [Fortlaufende Zahl pro Thread]", z. B. "Thread 2 - 4", "Thread 2 -- 5", "Thread 1 -- 3", "Thread 2 -- 6", und so weiter. Drei Threads lesen die Strings aus, entfernen sie aus der Schlange und geben sie auf der Kommandozeile aus.

Hinweise:

- Es darf weder Race Conditions noch Deadlocks geben.
- Nehmen Sie das Mutexing selbst vor.
- Achtung: bei manchen Sprachen ist die FIFO-Schlange selbst schon thread-sicher. In diesem Fall äußern sich Fehler beim Mutexing womöglich nicht.
- In der Praxis treten solche *Producer-Consumer-Szenarien* häufig auf, z. B.

wenn ein Sensor in einem selbstfahrenden Auto Hindernisse meldet, die von der Steuerungslogik abgearbeitet werden müssen.

Grundsätzliche Idee:

- **verwende eine Listendatenstruktur (oder schreibe sie selbst)**
- **zwei Funktionen `insert_into_list()` und `get_from_list()`, die zunächst Einträge einfüllen oder entfernen**
- **ein Mutex, der von diesen beiden Funktionen gleich nach dem Aufruf belegt und vor dem Rücksprung wieder freigegeben wird**

t) Angenommen, die Producer-Threads in der vorangehenden Teilaufgabe erzeugen sehr schnell Strings, während die Consumer-Threads sehr langsam Strings aus der Queue entnehmen.

Was für ein Problem ergibt sich? Ist das ein Deadlock? Wie lässt sich das Problem in der Praxis lösen?

Hinweis: Sie können das Problem im Code der vorigen Aufgabe nachstellen, indem Sie die Consumer-Threads nach jedem Verbrauchen mit einer `sleep`-Anweisung für eine hinreichend lange Zeit warten lassen.

Die Schlange mit Einträgen läuft voll. Irgendwann sind so viele Einträge in der Schlange, dass der Arbeitsspeicher des Computers nicht mehr genügt. Das ist kein Deadlock, sondern eine andere Art des "Verhungerns", ähnlich wie es beim Thread-Scheduling auftritt. In der Praxis könnte man z. B. die Größe der FIFO-Schlange beschränken, die Producer legen sich dann schlafen, bis wieder Platz in der FIFO-Schlange ist.

Aufgabe 2: Philosophen-Problem

Die folgende Aufgabe ist auch als Philosophen-Problem bekannt:

Vier Philosophen treffen sich, um ihren wichtigsten Tätigkeiten nachzugehen: Essen und denken. Da sie aber etwas zerstreut sind, vergisst jeder eines seiner zwei Esstäbchen zu Hause. Die Küche bringt zwar jedem der Philosophen einen ständig vollen Teller Reis, davon essen kann ein Philosoph jedoch nur, wenn er zwei Stäbchen zur Verfügung hat. Also einigen sie sich darauf, dass jeder sein Stäbchen auf seine rechte Seite legt. Da sie an einem runden Tisch sitzen, hat so jeder von ihnen zwei Stäbchen neben sich liegen. Es werden folgende Regeln ausgemacht:

- Ein Philosoph, der Hunger hat, nimmt sich erst sein linkes Stäbchen, dann sein rechtes, und isst, bis er satt ist. Danach legt er beide Stäbchen wieder hin und denkt, bis er wieder hungrig wird.
- Das Aufheben eines Stäbchens erfolgt mit der gebotenen Höflichkeit: Exakt ein Philosoph greift zu einer bestimmten Zeit nach einem Stäbchen, alle

anderen halten sich zurück und bekommen das Stäbchen nicht.

- Ein einmal aufgenommenes Stäbchen gibt man nicht wieder her, bis man sich vorest satt gegessen hat.

a) Nach drei Wochen sind alle Philosophen verhungert. Warum? Stellen Sie hierzu den Zustand der Philosophen vor dem Verhungern als Betriebsmittelgraph nach dem Diagramm der Vorlesung dar. Erläutern Sie, welche der vier Bedingungen für eine Verklemmungen erfüllt werden und warum.

Siehe Diagramm aus der VL von der Folie mit den vier Bedingungen, dabei sind die "Philosophen" die Threads (Linien im Diagramm) und die "Stäbchen" die Mutexe (bzw. Rechenressourcen, Kreise im Diagramm).

Exklusivität: Jedes Stäbchen kann nur von einem Philosophen gleichzeitig in der Hand gehalten werden.

Zusatzbelegung: Jeder Philosoph nimmt ein weiteres Stäbchen, während er ein Stäbchen exklusiv in der Hand hält.

Zyklus: siehe das Diagramm: P1 hat S1 und will das belegte S2, P2 hat S2 und will das belegte S3, P3 hat S3 und will das belegte S4, P4 hat S4 und will das belegte S1.

keine externe Freigabe: jeder Philosoph behält alle Stäbchen, bis er fertig ist mit dem Essen, es gibt keine andere Freigabe

b) Nach dem Debakel aus Teil a) werden neue Philosophen eingesetzt. Sie wollen verhindern, dass auch diese verhungern, und haben folgende Optionen:

- Einer der neuen Philosophen ist so geschickt, dass er beide Essstäbchen gleichzeitig greifen kann (und das nur tut, wenn auch beide gerade daliegen).

ja, löst das Problem -- Entweder der Philosoph bekommt beide Stäbchen und wird dann mit dem Essen fertig, wonach kein Zyklus mehr auftritt, oder der Philosoph muss auf eines der Stäbchen warten, lässt dabei aber alle Ressourcen unbelegt, wodurch auch kein Zyklus mehr auftritt.

- Sie setzen einen Philosophen in die Runde, der sich – wenn er länger als eine Stunde auf ein Stäbchen wartet – ein eventuell erworbenes Stäbchen zurücklegt und kurz mit seinem Hunger lebt, ehe er es neu versucht.

Hier gibt es eine externe Freigabe des Mutex während des Wartens durch den Philosophen und damit kein Deadlock mehr.

- Sie setzen einen Assistenten ein, der alle Stäbchen verwaltet und sie auf Anfrage paarweise an die Philosophen ausgibt.

Keine Zusatzbelegung mehr, da jeder Philosoph die Stäbchen gleichzeitig bekommt.

- Sie wählen diesmal nur Philosophen einer konkurrierenden Denkschule: Diese nehmen sich immer zuerst das rechte Stäbchen, dann das linke.

Das Deadlock kann immer noch auftreten, der Kreis aus dem Diagramm hat jetzt nur einen anderen Umlaufsinn, aber alle 4 Bedingungen bleiben erfüllt.

- Einer der Philosophen lebt nach dem Motto „mens sana in corpore sano“ und ist so kräftig, dass er seine Nachbarinnen zur Freigabe ihrer Stäbchen zwingen kann. Nach Essen und Rückgabe der Stäbchen geht er stets erst mal eine Runde joggen.

Externe Mutexfreigabe und Auflösung des Zyklus, kein Deadlock mehr.

Bei welchen dieser Optionen läuft die Runde wieder Gefahr zu verhungern (und warum)? Falls nicht, welche der Bedingungen für Verklemmungen sind nun nicht mehr erfüllt? Legen Sie (wie immer) Annahmen ausreichend dar und formulieren Sie präzise!

Aufgabe 3: Ex-Klausuraufgabe

In der Finanzbuchhaltung grauezellen arbeiten drei Bürokräfte *Alice* (A), *Bob* (B) und *Cheryl* (C) gleichzeitig an Unternehmensdokumenten. Konkret müssen regelmäßig zwei besonders wichtige Dokumente bearbeitet werden: die *Personalliste* (P) und der *Quartalsbericht* (Q). Beide Dokumente sind Dateien aus einer handelsüblichen Bürosoftware (z. B. Excel-Dateien). Jede Bürokraft muss hin und wieder jedes Dokument bearbeiten.

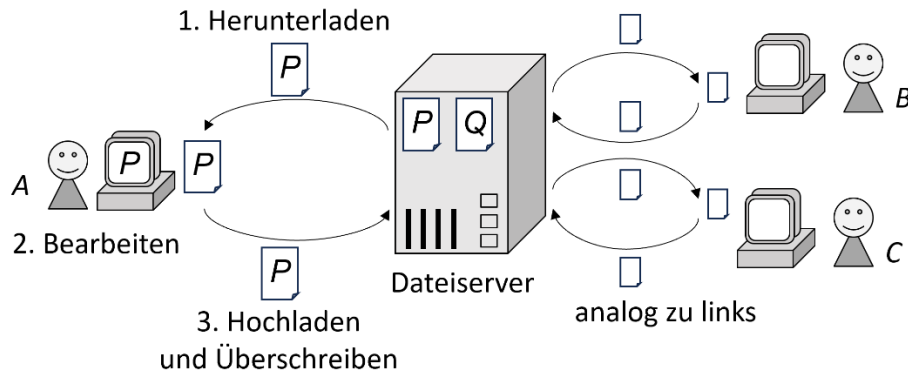
Die Dokumente liegen im aktuellen Stand auf einem Dateiserver, der über das Intranet des Unternehmens zu erreichen ist. Auf dem Dateiserver können die Dokumente aber nicht direkt bearbeitet werden, weil die Bürosoftware dort nicht läuft.

Zur Bearbeitung geht jede Bürokraft daher nach dem Vorgehen V_1 vor:

1. Die Bürokraft verbindet sich mit dem Server und lädt den aktuellen Stand des Dokuments vom Dateiserver auf den eigenen Bürorechner herunter.
2. Die Bürokraft trennt die Verbindung zum Dateiserver und bearbeitet im Anschluss das Dokument lokal auf dem eigenen Rechner.
3. Die Bürokraft verbindet sich zum Server und lädt die fertig bearbeitete Datei dort hoch. Die bearbeitete Datei überschreibt die vorherige Version.

Insgesamt können die Bürokräfte damit zeitgleich die Dateien bearbeiten, z. B. kann A die Datei P bearbeiten, während B die Datei Q bearbeitet.

Der Gesamtzusammenhang ist in folgender Abbildung illustriert:



Hinweise:

- Dass A im Diagramm **P** bearbeitet, ist nicht zwingend so, sondern nur ein Beispiel. Genauso gut könnte A die Datei **Q** bearbeiten.
- Da das **reine** Herunterladen, Hochladen und Überschreiben äußerst wenig Zeit braucht (wenige Millisekunden), sei angenommen, dass sich diese Vorgänge **nie** zeitlich überlappen. Es wird z. B. **niemals** zugleich eine Datei heruntergeladen und überschrieben. Gleichzeitiges Bearbeiten ist aber möglich.
- Das Intranet ist zuverlässig genug, dass alle Hochlade- und Herunterlade-Vorgänge auf jeden Fall gelingen. Es gibt keine Verbindungsabbrüche. Der Server ist ebenfalls so zuverlässig, dass alle Dateioperationen immer gelingen.

a) Im praktischen Betrieb merkt man schnell, dass das obige Vorgehen V_1 ein großes Fehlerpotential birgt. **Erklären Sie** mit Ihrer Kenntnis der Betriebssysteme an einer konkreten Abfolge von Hoch- und Herunterlade-Vorgängen der Bürokräfte A, B und C, **was schief gehen kann**, und **geben Sie ein Fachwort für das Problem an**.

Das wäre eine Race Condition: Wenn erst A die Datei P herunterlädt, dann verändert, dann lädt B die Datei herunter, ändert sie, A lädt hoch, dann B, damit überschreibt B die Änderung von A. Die Race Condition äußert sich nur bei geeigneter Reihenfolge, würde im Beispiel von gerade B erst nach dem Hochladen von A selbst herunterladen und die Bearbeitung mit Hochladen abschließen, würde man von der Race Condition nichts bemerken. Trotzdem ist die Race Condition vorhanden -- Race Conditions sind eine Eigenschaft eines parallelen Algorithmus und nicht eine Fehleräußerung!

b) Um das Problem zu lösen, schlägt die IT ein komplexeres Vorgehen V_2 vor:

1. Sobald ein Dokument heruntergeladen wird, markiert der Dateiserver das Dokument als "in Bearbeitung". Ein so markiertes Dokument kann zunächst von keiner weiteren Bürokraft mehr heruntergeladen werden. Versuche ergeben den Hinweis, zu warten, bis die Bearbeitung abgeschlossen ist.
2. Die Bürokraft, welche das Dokument heruntergeladen hat, bearbeitet **ausschließlich** jenes Dokument am Rechner fertig und lädt das Ergebnis hoch. Dieser Gesamtvorgang solle als Annahme **immer** irgendwann fertig sein.

3. Nach dem Hochladen wird das serverseitige Dokument automatisch ersetzt, die Kopie auf dem Rechner der Bürokräft automatisch gelöscht und zuletzt die Markierung "in Bearbeitung" automatisch entfernt.

Geben Sie ein **Fachwort** aus den Betriebssystemen an, welches diese Realisierung beschreibt. **Löst das Vorgehen V₂ das Problem von zuvor?** Begründen Sie!

Jetzt ist ein Mutex vorhanden: eine Belegungsflagge, die garantiert, dass nur ein Nutzer die Datei belegt und bearbeitet -- gleichzeitige Bearbeitungen sind mit dem Mutex nicht mehr möglich. Die Race Condition ist jetzt behoben.

- c) Nehmen Sie an, dass das komplexere Vorgehen V₂ der IT wie erklärt umgesetzt wird. Dummerweise zeigt sich jetzt eine andere Sorte Missverhalten:

Angenommen, Bürokräfte können beim Ändern einer beliebigen Datei bemerken, dass sie die andere Datei mitziehen müssen (z. B. um Verweise zu aktualisieren). Die entsprechende Bürokräft lädt in diesem Fall die zweite Datei vom Dateiserver ebenfalls herunter. Erst wenn die Änderungen an **beiden Dateien** abgeschlossen sind, darf die Bürokräft beide Dateien wieder auf den Dateiserver hochladen.

Was geht jetzt – als **Fachbegriff** der Betriebssysteme – schief? **Wie äußert sich das Problem? Erklären Sie** an einer konkreten Folge von Herunter- und Hochlade-Vorgängen der Bürokräfte A, B und C.

Durch die beiden unterschiedlichen Dateien liegt jetzt ein Deadlock vor: Falls A die Datei P herunterlädt, B die Datei Q, dann sind P und Q gesperrt. Würde A als nächstes die Datei Q brauchen (ehe er hochladen und den Mutex zu P freigeben kann) und B als nächstes die Datei P (wieder ehe er hochladen und den Mutex zu Q freigeben kann), dann warten A und B endlos aufeinander.

Wir sehen hier die 4 Bedingungen für ein Deadlock:

Mutex -> exklusive Belegung der Dateien

Zusatzbelegung -> mehrere Beteiligte halten den Mutex zu einer Datei belegt und wollen den Mutex zu einer weiteren Datei belegen

Zyklus -> es liegt eine kreisförmige Wartesituation vor: A hält P und will Q (belegt von B), B hält Q und will P (belegt von A)

keine externe Freigabe -> der Mutex wird ausschließlich durch das Hochladen mit dem Ende der Gesamtbearbeitung mehrerer Dateien freigegeben (aber nicht z. B. durch ein Zeitlimit für die Bearbeitung)

- d) Nehmen Sie weiterhin die komplexere Realisierung der IT nach V₂ an. Nehmen Sie zudem an, dass die Bearbeitung von Datei P nie den Anlass gibt, auch Datei Q zu bearbeiten. Es kann aber jeder Bürokräft passieren, dass bei der Bearbeitung von Datei Q die Datei P ebenfalls nachgezogen werden muss. In diesem Fall lädt die Bürokräft erst Q, dann P herunter, abschließend beide Dateien gemeinsam hoch.

Kann jetzt etwas schiefgehen? Begründen Sie mit einer Bedingung der VL.

Hier gibt es zwar noch eine Zusatzbelegung (von Q nach P), aber keinen Zyklus mehr: die Bearbeitung der Datei P wartet nie auf einen Mutex, damit wird die Datei P irgendwann immer freigegeben, damit erhält die Bearbeitung von Q (sofern nötig) immer irgendwann Zugriff auf P und wird damit selbst immer irgendwann fertig